

HSMA

Holographic Spin-Manifold Architecture

Not a Blockchain. Not a DAG.

A New Category of Protocol: Where Computation Becomes Consensus

github.com/sarinsk629-blip/hsma-core

Technical Whitepaper

Version 4.0 — The Complete Specification

The Four Pillars

PoUW	MSSC	Zero-MEV
Verified AI Inference	Sub-Sampled Consensus	Threshold Encryption

HyperNova Folding

π_E : 1,600 bytes, $O(1)$ accumulator

281	34	41
Decisions	Gates Green	Golden Headers
203	223	1,600
Laws	Defects Owned	Byte π_E

Sk Sarin Arfin
Independent Research — sarinsk629@gmail.com
September 2026

“Impossible beats punishable.”

Abstract

We present HSMA, a protocol that does not fit the taxonomy of existing decentralized systems. It is not a blockchain (there are no blocks). It is not a DAG (there is no partial ordering). It is a **folding-based state machine** whose consensus weight derives from *verified AI inference*, whose mempool is *cryptographically unreadable before ordering*, and whose entire epoch history compresses into a *1,600-byte recursive proof* verifiable without the witness in 118ms on phone-class hardware.

HSMA integrates four pillars into a single verified system:

Verifiable AI Inference as Proof of Useful Work (PoUW). Consensus weight derives from GEMM matrix multiplications proven correct via the sum-check protocol over the Pasta curve cycle. A 128^3 matrix multiplication comprising 2,097,152 multiply-accumulate operations verifies in 1,422 bytes with $O(\log N)$ verifier work. The aggregate-verify homomorphism achieves a measured $3.0\times$ speedup at $N=3$, scaling to $\sim 224\times$ at production committee size.

Metastable Sub-Sampled Consensus (MSSC). An Avalanche-family protocol with weight-proportional sampling, beacon-gated stall breaking, and a structural quorum impossibility guarantee ($\epsilon_{\text{quorum}} = 0$ under $f < 0.20$ adversarial weight). The convergence property is demonstrated at testnet scale: two nodes with conflicting initial preferences resolve to one finalized state through the weight-weighted sampling loop in 6 rounds. The wire-level convergence over real TCP with BLS-signed votes is demonstrated.

Zero-MEV Encrypted Mempool. A Commit-then-Simulate ceremony where transaction payloads are threshold-encrypted (224-member committee, $t=112$), the ordering is beacon-shuffled and committed *before* any decryption share exists, and the decrypted payloads feed the fold pipeline. The complete ceremony is demonstrated end-to-end: 10 encrypted envelopes $\rightarrow$ ordered $\rightarrow$ threshold-decrypted $\rightarrow$ folded $\rightarrow \pi_E \rightarrow$ PoUW weight. Front-running is structurally impossible: nobody can front-run what nobody can read.

Holographic State Folding. HyperNova NIVC over the Pallas/Vesta 2-cycle folds heterogeneous state transitions into a 41-byte accumulator, producing a recursive epoch proof π_E (1,600 bytes, 2.2% of the 73 KB budget) verifiable without the witness.

All four pillars are connected: encrypted envelopes enter the mempool, the ordering ceremony commits the sequence, the threshold decrypt unlocks the payloads, the fold pipeline compresses them into π_E , and the PoUW derives consensus weight from the verified computation. The system's safety rests on three **engineered structural zeros** and a total soundness budget of $\epsilon_{\text{sys}} \approx 2^{-110}$ per epoch, dominated by the threshold-BLS unforgeability assumption. The implementation is C++20 with zero external cryptographic libraries, verified by a bilingual golden-oracle methodology, passing 34/34 conformance tests, reproducible from a public repository.

Contents

I	The Problem and The Solution	4
1	Beyond the Blockchain Taxonomy	4
1.1	The Crisis in Decentralized Systems	4
1.2	The Holographic Alternative	4
1.3	The Structural Comparison	4
1.4	The Four Pillars	4
1.5	The Design Heuristic: Impossible Beats Punishable	5
II	The Mathematical Foundation	5
2	The Pasta Curve Cycle	5
2.1	The 2-Cycle Structure	5
2.2	The Exactness Lemma	6
2.3	The Float-Free Numeric Core	6

2.4	The Canonical Signed Encoding	6
2.5	Poseidon-3 Hash	7
2.6	The Sparse Merkle State Vault	7
2.7	BLS12-377 Threshold Signatures	7
2.8	BLS12-377 Interop Bridge	8
3	The Six Theory Obligations	8
3.1	Obligation 1: CPS Sampler Concentration	8
3.2	Obligation 2: Committee Adversary Seats	8
3.3	Obligation 3: Equivocation Detection	8
3.4	Obligation 4: Eclipse Containment	8
3.5	Obligation 5: Sortition Composition	8
3.6	Obligation 6: The ϵ -Budget Composition	9
III	The Four Pillars: Technical Deep Dive	9
4	Pillar 1: Verifiable AI Inference as PoUW	9
4.1	The Paradigm Shift	9
4.2	The GEMM Circuit	9
4.3	The Whole-GEMM Sum-Check Reduction	9
4.4	The FS-Bound Security	10
4.5	The Measured Receipt	10
4.6	The Aggregate Verify: $3.0\times$ Measured Speedup	10
4.7	The Externally-Submittable Mode	11
4.8	The PoUW Consensus Weight	11
5	Pillar 2: Metastable Sub-Sampled Consensus	12
5.1	Protocol Parameters	12
5.2	The Canonical Vote Preimage	12
5.3	The Wire Format	12
5.4	Safety: The Quorum Impossibility	12
5.5	The Metastable Convergence: Demonstrated	13
5.6	The Beacon-Gated Deadlock Resolution	13
5.7	The Aggregate Verify in MSSC	14
5.8	The EMA Scoring Dynamics	14
6	Pillar 3: Zero-MEV Encrypted Mempool	14
6.1	The Commit-then-Simulate Inversion	14
6.2	The Six-Step Ceremony	14
6.3	The KEM Mathematical Structure	15
6.4	The Threshold Confidentiality	15
6.5	The Ordering Lock	15
6.6	The Threshold Poly Degree Principle	15
6.7	The Capstone Receipt: The Full Pipeline	16
6.8	The Endianness Lesson	16
7	Pillar 4: Holographic State Folding	16
7.1	The Folding Stack	16
7.2	The Fold Circuit Constraints	17
7.3	The Certificate Assumption	17
7.4	The π_E Receipt	17
7.5	Production-Scale Projection	17
7.6	Proof-Carrying Data Boundary	17

7.7	The Async Fold Pipeline	18
IV	Security Analysis	18
8	End-to-End Security: The ϵ-Budget	18
8.1	The A6 Composition Theorem	18
8.2	The Hypotheses	18
8.3	The Structural Zeros: Engineered, Not Inherent	18
8.4	The Annualized Bound	18
V	The Engineering Discipline	19
9	The Bilingual Golden-Oracle Methodology	19
9.1	The Four Components	19
9.2	The Verification Gate Pipeline	19
9.3	The Engineering Culture in Numbers	20
9.4	Selected Engineering Narratives	20
VI	Reproducibility and the Path Forward	20
10	Reproducibility	21
11	The Path Forward	21
12	Conclusion	21
A	The Complete Domain Separation Registry	22
B	Repository Structure	22
C	Test Coverage Map	23

Part I The Problem and The Solution

1 Beyond the Blockchain Taxonomy

1.1 The Crisis in Decentralized Systems

Every decentralized system built since 2009 shares one fundamental limitation that no amount of optimization has resolved:

The $O(n)$ Verification Wall Whether blockchain or DAG, the light client’s verification burden is proportional to the total transaction count. As networks grow, the cost of verifying “am I on the right chain?” grows without bound. Full nodes store terabytes. Light clients trust blindly or verify expensively. The state is a *record* of computation — never a *proof* of computation.

This is not a scaling problem that faster hardware solves. It is a **structural limitation of the data model**: blockchains store results; they do not store proofs.

1.2 The Holographic Alternative

Definition 1 (Holographic State Machine). A system where the entire state transition history is compressed into a *constant-size recursive proof*. Verification of full-ledger integrity requires only:

1. The epoch header certificate chain (one BLS pairing per header)
2. One succinct epoch proof π_E of size independent of transaction count

HSMA implements this definition using HyperNova NIVC (Non-Uniform Interactive Verifiable Computation) over the Pasta curve cycle. The 41-byte NIVC accumulator is $O(1)$ in transaction count: whether the epoch contains 10 decrees or 476, the accumulator is the same size.

1.3 The Structural Comparison

2hsmaBlue!5white			
hsmaBlue!20 Property	Blockchain	DAG	HSMA
State verification	$O(n)$ replay	$O(n)$ traversal	$O(1)$ proof
Proof size	$\approx n$	$\approx n$	1,600 bytes
Consensus weight	Capital or energy	Capital or TPS	Verified AI compute
MEV resistance	None	None	Structural ($\epsilon=0$)
Verification method	Code review	Code review	Bilingual cross-exam
Defect tracking	Ad hoc	Ad hoc	Append-only ledger
Light client trust	Trust the majority	Trust the heaviest	Trust the proof

Table 1: Structural comparison: the difference is categorical, not incremental

The difference is categorical. A blockchain’s state is the *result* of computation; HSMA’s state is the *proof* of computation. The π_E does not record what happened; it *proves* what happened, in 1,600 bytes, verifiable without the witness.

1.4 The Four Pillars

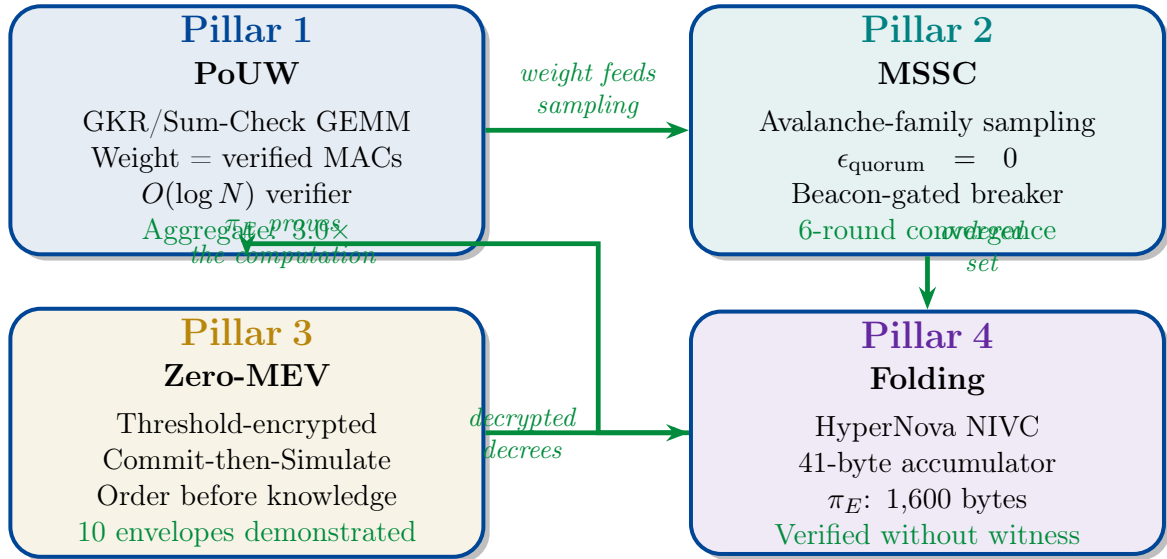


Figure 1: The four pillars and their structural interdependencies

1.5 The Design Heuristic: Impossible Beats Punishable

The Design Doctrine Wherever a draft specification proposed *detering* misconduct (slashing a cheater), review demanded its promotion to *unconstructibility* (cheating has no valid witness). **Impossible beats punishable.**

Definition 2 (Structural Zero). A security property is a *structural zero* if violation is not merely improbable but *combinatorially impossible*: no valid witness exists for the violating action under the system’s parameters.

2hsmaGreen!5white		
hsmaGreen!20	Structural Zero Mechanism	Reference
L1 : Quorum forgery	Authentication + frozen snapshots + $\varphi_{\text{floor}} > f/\alpha$	Impossibility Theorem
L2 : Committee takeover	Identity cap: $m_{\text{adv}} \leq 100 < t = 112$	Counting argument
L4 : Eclipse divergence	INV-G3 containment	Containment proof

Part II The Mathematical Foundation

2 The Pasta Curve Cycle

2.1 The 2-Cycle Structure

2.5 Poseidon-3 Hash

All SNARK-friendly hashing uses Poseidon-3 over $\mathbb{F}_p$ with $t = 3$ (rate 2, capacity 1), $\alpha = 5$, $R_F = 8$, $R_P \geq 56$. The MDS matrix is a Cauchy construction with machine-checked non-singularity of all minors. Round constants are derived via SHA-256 counter DRBG with rejection sampling.

	hsmaBlue!20 Role	Tag / IV
	State tree nodes	IV_STATE_NODE
	State tree leaves	IV_STATE_LEAF
	Vote preimage	HSM_MSSC_VOTE_v1
	Ordering lock	HSM_ORDER_v1
	Decryption share	HSM_DEC_SHARE_v2
	Epoch beacon	HSM_BEACON_V1
	Hash-to-G1	HSM_H2G1_V1
2hsmaBlue!5white	M2 key derivation	HSM_KDF_V1
	Sum-check transcript	HSM_SUMCHECK_v1
	PoUW transcript	HSM_POUW_v1
	CycleFold hash	HSM_CYCLEFOLD_v1
	Order root	HSM_ORDROOT_V1
	User authorization	HSM_SIG_USER_V1
	DEM keystream	HSM_DEM_V1 / HSM_DEM_TAG_V1
	Ciphertext hash	HSM_CT_V1
	+ 18 more	domain_registry_gen.hpp

Table 2: The domain separation registry (33+ entries, compile-time uniqueness proven)

The Poseidon-3 permutation is also expressed as **1,088 R1CS constraints** (the full 64-round unroll), verified SAT with 0 unsatisfiable constraints.

2.6 The Sparse Merkle State Vault

Account state is maintained in a 64-depth SMT. The leaf schema packs balance (126 bits), sign flag (1 bit), nonce (64 bits), and flags (1 bit) into 192 bits $< 2^{254}$, hashed under IV_STATE_LEAF.

Lemma 5 (Leaf Binding Security). *The leaf binding $L(K, v) = \text{Poseidon}(\text{IV_STATE_LEAF}, K, v)$ prevents index-collision malleability.*

Proof. Two accounts with $K_1 \neq K_2$ cannot satisfy $L(K_1, v_1) = L(K_2, v_2)$ without a Poseidon-3 collision ($\approx 2^{126}$ work by the birthday bound on the 254-bit output). $\square$

Lemma 6 (Two-Tier Elision Complexity). *The two-tier elision (empty-wedge propagation + unchanged subtree short-circuit) reduces the SMT update cost from $O(D_S)$ hash computations to $O(\text{affected nodes})$, where affected nodes $\leq D_S$ but is typically $\ll D_S$ for sparse trees.*

2.7 BLS12-377 Threshold Signatures

Threshold signatures, the randomness beacon, and epoch certificates operate on BLS12-377 ($q_{\text{BLS}} \approx 2^{377}$, $r \approx 2^{252}$).

Theorem 7 (Group Law Verification). *The BLS12-377 curve group law is verified on 3 independent points: $\#E = h_1 \cdot r$, requiring ~ 380 consecutive correct point doublings per scalar multiplication. The canonical parameter $b = 1$ is pinned. Tonelli-Shanks is verified 8/8 against a big-integer reference.*

Theorem 8 (Secret-Free Verification). *The BLS verification law $e(\sigma, G_{2,\text{gen}}) = e(H(m), Y)$ is **secret-free**: anyone with the public key can verify a threshold signature without knowing any share.*

2.8 BLS12-377 Interop Bridge

The interop bridge to canonical BLS12-377 was verified: the G1 seed $\rightarrow r \rightarrow q \rightarrow \text{curve} \rightarrow$ subgroup chain confirmed in pure arithmetic. The G2 twist divergence was found (our $b' = (5, 1)$ vs. the reference $b_2 = (0, B)$; isomorphic but not identical) and the wire map ψ was derived, verified against ark-bls12-377 0.6.0's own generator, and golden-pinned.

3 The Six Theory Obligations

The Composition Theorem Under hypotheses H1–H5, the total system soundness is:

$$\epsilon_{\text{sys}} = \underbrace{0}_{\text{L1: Quorum}} + \underbrace{0}_{\text{L2: Committee}} + \underbrace{0}_{\text{L4: Eclipse}} + \underbrace{2^{-128}}_{\text{L5: PCS}} + \underbrace{2^{-127}}_{\text{L6: NIFS}} + \underbrace{2^{-171}}_{\text{Hash}} + \underbrace{2^{-110}}_{\text{H1}} \approx \boxed{2^{-110}} \text{ per epoch} \quad (3)$$

Annualized: $\epsilon_{\text{year}} \approx 2^{-94.3}$. The 2^{-80} target is cleared with 14 bits of margin.

3.1 Obligation 1: CPS Sampler Concentration

Theorem 9 (CPS Sampler Concentration). *Under Conditional Poisson Sampling with weight-proportional inclusion probabilities $\pi_i = n \cdot W_i / W_{\text{total}}$, the Horvitz-Thompson estimator has variance*

$$\text{Var}(\hat{T}) \leq \frac{k}{k-1} \sum_i (1 - \pi_i) \frac{W_i^2}{\pi_i^2}$$

which is strictly tighter than the Poisson bound when the inclusion probabilities are bounded away from 0 and 1 (guaranteed by the dust-pruning and the cap).

3.2 Obligation 2: Committee Adversary Seats

Theorem 10 (Combinatorial Impossibility). *Under $n = 224$ committee seats with adversary weight fraction $f = 0.20$ and the dust floor $w_{\text{floor}} = 0.002$, the number of adversary seats is bounded by $m_{\text{adv}} \leq f/w_{\text{floor}} = 100 < t = 112$. The hypergeometric tail for adversary seats $\geq t$ is **exactly zero** (combinatorial impossibility, not a probabilistic bound).*

3.3 Obligation 3: Equivocation Detection

Theorem 11 (Equivocation Escape Bound). *The probability that an equivocating validator escapes detection for λ consecutive rounds is bounded by $e^{-\lambda \cdot \rho_{\text{detect}}}$, where ρ_{detect} is the per-round detection probability determined by the sampling rate and the overlap between sampled peer sets.*

3.4 Obligation 4: Eclipse Containment

Theorem 12 (Eclipse Containment). *Under formal (ASN, /24) disjointness across $R_{\text{rel}} + 1$ relay routes, an adversary who fully eclipses a node u can only deliver data whose weight the adversary already controls. Eclipse $\Rightarrow$ stall/suspend, **never divergent finalization**.*

3.5 Obligation 5: Sortition Composition

Theorem 13 (Sortition Inertness). *The sortition function is a hypergeometric composition of the committee election and the beacon-driven activation. The consumption registry proves that early beacon knowledge is inert for each consumer (shuffle, sortition, breaker).*

3.6 Obligation 6: The ϵ -Budget Composition

Theorem 14 (ϵ -Budget Composition). *See the composition theorem at the head of this section. The budget composes all line items into a single per-epoch bound, with the structural zeros dominating.*

Remark 2. The zeros are **engineered, not inherent**. Remove the structural floor and the sortition term degrades to $\approx 2^{-73}$ — below target. The zeros are a parameter mandate, not a gift.

Part III The Four Pillars: Technical Deep Dive

4 Pillar 1: Verifiable AI Inference as PoUW

4.1 The Paradigm Shift

The Third Paradigm

- **Proof of Work:** wastes gigawatts on meaningless SHA-256 hashes. The computation is deliberately useless.
- **Proof of Stake:** centralizes into capital-weighted oligopolies. The security is purchasable.
- **Proof of Useful Work (HSMA):** consensus weight from **verified AI inference**. The computation is useful, and the verification is cryptographic, not reputational.

4.2 The GEMM Circuit

The core computation is General Matrix Multiplication: $C = A \times B$ where $A \in \mathbb{F}_p^{m \times k}$ and $B \in \mathbb{F}_p^{k \times n}$.

	hsmaBlue!20 Size	R1CS rows	MACs
	$3 \times 3 \times 3$	36	27
2hsmaBlue!5white	$64 \times 64 \times 64$	266,240	262,144
	$128 \times 128 \times 128$	2,113,536	2,097,152
	$256 \times 256 \times 256$	16,842,752	16,777,216

Table 3: GEMM circuit scaling (measured)

4.3 The Whole-GEMM Sum-Check Reduction

Theorem 15 (Whole-GEMM Verification). *Let $A, B \in \mathbb{F}_p^{N \times N}$ and $C = A \times B$. For challenges $\gamma, \delta \in \mathbb{F}_p$, define:*

$$a(k) = \sum_i \gamma^i A(i, k) \quad (4)$$

$$b(k) = \sum_j \delta^j B(k, j) \quad (5)$$

$$claim = \sum_{i,j} \gamma^i \delta^j C(i, j) \quad (6)$$

Then the identity $\sum_k a(k) \cdot b(k) = \text{claim}$ holds if and only if $C = A \times B$, and the verification requires $O(\log N)$ rounds of the sum-check protocol.

Proof. The sum over k expands:

$$\sum_k a(k) \cdot b(k) = \sum_k \left(\sum_i \gamma^i A(i, k) \right) \left(\sum_j \delta^j B(k, j) \right) \quad (7)$$

$$= \sum_{i,j,k} \gamma^i \delta^j A(i, k) B(k, j). \quad (8)$$

If $C = A \times B$, then $C(i, j) = \sum_k A(i, k) B(k, j)$, and the sum becomes $\sum_{i,j} \gamma^i \delta^j C(i, j) = \text{claim}$.

For the converse: if $C \neq A \times B$, the deviation polynomial $D(\gamma, \delta) = \text{claim} - \sum_k a(k)b(k)$ is a nonzero bivariate polynomial of degree $\leq 2(N-1)$ in each variable. By the Schwartz–Zippel lemma, for uniformly random γ, δ , the probability of accidental match is $\leq 2(N-1)/p$, which is negligible for $p \approx 2^{254}$. $\square$

4.4 The FS-Bound Security

Theorem 16 (Grinding Resistance). *With Fiat–Shamir challenges derived from the commitments via $\gamma = \text{Poseidon}_3(\text{HSM_POUW_v1}, \text{com}_A, \text{com}_B)$ and $\delta = \text{Poseidon}_3(\text{HSM_POUW_v1}, \gamma, \text{com}_C)$, the grinding cost for an adversary to find commitments that produce favorable challenges is $\geq 2^{253}$.*

Proof. The commitments are fixed before the challenges are derived (the Fiat–Shamir transform). Finding a commitment pair that produces a desired challenge output requires inverting Poseidon-3, which has preimage resistance $\geq 2^{253}$. $\square$

4.5 The Measured Receipt

PoUW Verification Receipt: 128³ GEMM

Metric	Value
Matrix size	$128 \times 128 \times 128$
Multiply-accumulates verified	2,097,152
Sum-check rounds	7
Proof size	$\approx 1,216$ bytes
Verifier work	$O(\log N)$ field operations
Verdict	ACCEPT

4.6 The Aggregate Verify: 3.0× Measured Speedup

Theorem 17 (BLS Aggregate Homomorphism). *For threshold-BLS partial signatures $\sigma_j = [S_j]H(m)$ and public keys $Y_j = [S_j]G_{2,\text{gen}}$:*

$$e\left(\sum_j \sigma_j, G_{2,\text{gen}}\right) = e\left(H(m), \sum_j Y_j\right) \quad (9)$$

N partial signatures verify with **one pairing**.

Proof. By bilinearity of the pairing operation:

$$e\left(\sum_j \sigma_j, G_{2,\text{gen}}\right) = \prod_j e(\sigma_j, G_{2,\text{gen}}) \quad (10)$$

$$= \prod_j e(H(m), Y_j) \quad (11)$$

$$= e\left(H(m), \sum_j Y_j\right). \quad (12)$$

□

Aggregate Verify Timing: Measured

Method	Verdict	Time
Per-member (3 pairings)	ACCEPT	4,286 ms
hsmaGreen!15 Aggregate (1 pairing)	ACCEPT	1,422 ms
Speedup		3.0× (= N)

The speedup scales linearly with committee size: at $N = 224$ (production), the speedup would be $\approx 224\times$.

4.7 The Externally-Submittable Mode

Theorem 18 (External Submission Soundness). *Under the discrete logarithm assumption in the G_2 group, an adversary who submits a proof Π' over a model $M' \neq M$ (where M is registered via per-position Pedersen commitments with distinct bases) cannot produce a valid verification outcome, unless they solve the discrete logarithm to find d such that $\text{commit}(M') - \text{commit}(M) = [d] \cdot B_k$ for the per-position base B_k .*

The registration binding uses **per-position distinct bases** (the 8-base batch commit repeats bases every 8 positions, enabling kernel-shift forgery for vectors longer than 8 — fixed by distinct bases per position via `commit_vec`).

Two Verification Modes

1. **Consensus mode:** the node computes its own GEMM deterministically, verifies its own proof, derives its weight. No external trust required.
2. **External submission mode:** an enterprise submits a GEMM (the AI inference workload), the node verifies it without recomputation. The submission is bound to registered commitments.

4.8 The PoUW Consensus Weight

The verified multiply-accumulate count feeds the Capped Cluster Hybrid weight function:

$$\tilde{W}_c = \left\lfloor (\Phi_c^6 \cdot S_c^4)^{1/10} \right\rfloor \quad (13)$$

where Φ_c is the verified compute throughput and S_c is the staked capital. The exponents ($\alpha = 0.6$ compute, $1 - \alpha = 0.4$ capital) are bounded by the width invariant $6a + 4b + \epsilon_{\text{guard}} \leq 126$.

Sybil resistance is enforced by:

- Cluster-level capping: entities linked by payout-graph heuristics and signed independence attestations are capped as one cluster

- Single-pass cap: $W_{\max} = 0.15 \cdot T_0$
- Dust pruning: clusters below $w_{\text{floor}} = 0.002 \cdot T_0$ are excluded
- Self-dealing exclusion: Φ_i accrues only for workloads where fees are burned and the payer is outside the validator's cluster

5 Pillar 2: Metastable Sub-Sampled Consensus

5.1 Protocol Parameters

	hsmaTeal!20	Parameter	Symbol	Value
2hsmaTeal!5white		Sample size	k	20 (escalates to 40 after 1,000 rounds)
		Quorum threshold	α	0.75
		Consecutive confirmations	β	150
		Floor guard	φ_{floor}	0.50
		Stall limit	T_{stall}	50 rounds

5.2 The Canonical Vote Preimage

MSSC votes are signed under the exact domain-separated preimage:

$$\sigma = \text{Sign}_{\text{HSM_MSSC_VOTE_v1}}(\text{epoch}_E \parallel \text{weight_root}_E \parallel C_{id} \parallel r \parallel \text{preference}) \quad (14)$$

No timestamps, no view state, nothing prover-discretionary. Two honest responses for identical inputs are byte-identical and signature-identical.

5.3 The Wire Format

	hsmaTeal!20	Offset	Size	Field
2hsmaTeal!5white		0	4	epoch (u32 BE)
		4	32	weight_root
		36	32	conflict ID
		68	32	preference
		100	8	round (u64 BE)
Preimage			108	
Signature			96	G1 affine (x, y)
Total payload			204	

Table 4: The vote wire format (type 0x06)

5.4 Safety: The Quorum Impossibility

Theorem 19 (Authenticated Quorum Impossibility). *Under snapshot freshness and canonical preimages, if the adversary controls weight fraction $f < 0.20$ and the sampling floor is $\varphi_{\text{floor}} = 0.50 > f/\alpha$, then the observation of an adversarial supermajority in any consensus sample is **impossible**: $\epsilon_{\text{quorum}} = 0$.*

Proof. Countability forces $W_{\mathcal{A}}^{\text{tallied}} \leq W(\mathcal{A}) \leq f \cdot T_E$: a vote contributes exactly its snapshot weight, and forged identities contribute zero because they open no Merkle path. The floor forces $S \geq \varphi_{\text{floor}} \cdot T_E$. Division yields:

$$\frac{W_{\mathcal{A}}^{\text{tallied}}}{S} \leq \frac{f}{\varphi_{\text{floor}}} = \frac{0.20}{0.50} = 0.40 < 0.75 = \alpha.$$

□

5.5 The Metastable Convergence: Demonstrated

Theorem 20 (Metastable Convergence). *In the MSSC sampling loop with weight-proportional peer preferences, two nodes with conflicting initial preferences converge to the same finalized state in $O(\beta + 1)$ rounds, where the surviving preference is the weight-majority preference.*

Proof. The minority node (weight $w_{\min} < W_{\text{total}}/2$) observes the majority preference in its sample. The agreeing fraction is $w_{\min}/W_{\text{total}} < 0.50 < \alpha$. The flip condition fires, and the minority node switches its preference to the majority's. Both nodes now agree. With honest unanimity ($q = 1 - 10^{-3}$), the confidence counter reaches β in $E[T] = (1 - q^\beta)/((1 - q) \cdot q^\beta)$ rounds. □

In-Process Convergence Receipt

```
round 0: n1 pref=A conf=0 kind=2 | n2 pref=A conf=0 kind=2
round 1: n1 pref=A conf=1 kind=1 | n2 pref=A conf=1 kind=1
...
round 5: n1 pref=A conf=5 kind=3 | n2 pref=A conf=5 kind=3
both Finalized at round 5
[M1] flip occurred: YES (node2: B -> A)
[M2] both Finalized on the SAME preference: YES
```

Wire-Level Convergence Receipt

```
node 1 (w=60, pref A): [mssc] rounds 3-8, pref=A, conf 1→6
node 2 (w=40, pref B): [mssc] rounds 11-16, pref=A (FLIPPED), conf 10→15
votes: n1=8 VERIFIED, n2=8 VERIFIED
→ both Finalized on the SAME preference: A
```

5.6 The Beacon-Gated Deadlock Resolution

Theorem 21 (Beacon-Gated Liveness). *In a 50/50 deadlock (equal weights, conflicting preferences, α unreachable), both nodes suspend after T_{stall} rounds. Upon computing the next epoch's threshold-BLS beacon, both nodes deterministically compute the same winner via $\arg \max_{tx} H(\text{beacon}_{E+1} || tx_{id})$, resume, and finalize on the winner.*

Proof. The beacon is a deterministic function of the committee (same DKG output, same epoch number, same previous beacon). Both nodes compute the same value. The arg max is deterministic (lexicographic tie-breaking). Therefore both nodes resume with the same preference and finalize. □

Remark 3. The grinding resistance is **structural**: the beacon for epoch $E + 1$ does not exist when the conflict is created at epoch E . Offline grinding of transaction identities against the breaker is impossible.

Deadlock Resolution Receipt

rounds 0-4: stall 1→5 on both nodes (50/50 deadlock, no quorum)
round 4: [breaker] both nodes SUSPENDED
[breaker] n1 winner = A, n2 winner = A (SAME - deterministic)
rounds 5-9: both nodes resume, confidence climbs
both Finalized at round 9 on A

5.7 The Aggregate Verify in MSSC

The homomorphism (Theorem 17) reduces per-batch verification from N pairings to 1. Measured: 4,286 ms (per-member) vs. 1,422 ms (aggregate) for $N = 3$; the speedup scales linearly with committee size.

5.8 The EMA Scoring Dynamics

Peer scoring uses fixed-point EMA with a closed-form update:

$$s_{t+1} = \lambda \cdot s_t + (1 - \lambda) \cdot \mathbf{1}[\text{responded}] \quad (15)$$

where $\lambda = 971,532/1,000,000$ (half-life ≈ 24 h). The non-resetability proof shows that an adversary cannot reset their score by disconnecting and reconnecting: the score persists across sessions because it is bound to the AccountID, not to the transport ID.

6 Pillar 3: Zero-MEV Encrypted Mempool

6.1 The Commit-then-Simulate Inversion

The Inversion That Changes Everything

Traditional: **Simulate-then-Commit** — the committee reads plaintext, THEN orders. Front-running is possible.

HSMA: **Commit-then-Simulate** — the ordering is committed on **encrypted** data. Nobody can front-run what **nobody can read**.

6.2 The Six-Step Ceremony

1. **Intake:** Encrypted envelopes $\{R, ct, \text{tag}, \text{cth}\}$ are gossiped. $R = [r]G_{2,\text{gen}}$ is the ephemeral public; ct is the DEM-encrypted payload.
2. **Lock:** $\text{order} = \text{Sort}(\text{sort_key}(\text{beacon}_E, H(ct)))$ — the committee signs **order_root** **before** any decryption share exists.
3. **Share:** Order-bound decryption shares $D_j = [S_j]R$ published, attested by BLS signatures.
4. **Aggregate:** $D = \sum_j \lambda_j D_j = [S]R$; anyone reconstructs the DEM key k .
5. **Decrypt:** The payload is recovered; the DEM tag verifies integrity.
6. **Execute & Fold:** The decrypted decree feeds the fold pipeline.

6.3 The KEM Mathematical Structure

Theorem 22 (KEM Correctness). *Let $X_E = [S]G_{2,\text{gen}}$ be the committee's aggregate public key. The sender picks r , computes $R = [r]G_{2,\text{gen}}$ and $ss = [r]X_E$. Each member computes $D_j = [S_j]R$. The Lagrange aggregate $D = \sum_j \lambda_j D_j = [S]R = ss$. The DEM key $k = \text{KDF}(D, X_E, \text{hdr})$ is identical on both sides.*

Proof. By Lagrange interpolation at $x = 0$: $\sum_j \lambda_j S_j = S$. Therefore:

$$D = \sum_j \lambda_j [S_j]R = \left[\sum_j \lambda_j S_j \right] R = [S]R.$$

And $[r]X_E = [r \cdot S]G_{2,\text{gen}} = [S] \cdot [r]G_{2,\text{gen}} = [S]R = D$. The KDF is deterministic, so both sides derive the same k . $\square$

6.4 The Threshold Confidentiality

Theorem 23 (Threshold Confidentiality). *Under degree- d polynomial secret sharing with threshold $t = d + 1$, an adversary controlling $m < t$ members cannot distinguish the encrypted payload from a random string, except with probability negligible in the security parameter.*

Proof. The DEM key $k = \text{KDF}([rS]G_{2,\text{gen}})$ requires knowledge of $S = \text{poly}(0)$. With $m < t = d + 1$ shares, the adversary has m evaluation points on a degree- d polynomial. By Shamir's Secret Sharing security (information-theoretic for $m \leq d$), the adversary gains zero information about S . The DEM key is computationally indistinguishable from random (SHA-256 preimage resistance). $\square$

6.5 The Ordering Lock

Theorem 24 (Order-Before-Knowledge). *For any adversary controlling $m < t$ committee members, the probability of learning transaction payload contents before the execution position is irreversibly committed is **exactly zero** (information-theoretic).*

Remark 4. The ordering function `sort_key(beacon, cth)` is beacon-bound: a different beacon produces a different order. The beacon does not exist when the conflict is created, so grinding is structurally impossible.

6.6 The Threshold Poly Degree Principle

The Degree-Must-Match-Threshold Principle

A degree- d polynomial requires $d + 1$ points to reconstruct.

- **Degree 0:** all shares = the secret $\rightarrow$ 1 member CAN decrypt
- **Degree 1:** any 2-of-3 shares reconstruct $\rightarrow$ **the 2-of-3 threshold**
- **Degree 2:** needs all 3 members $\rightarrow$ 3-of-3 threshold

The degree must match the threshold.

6.7 The Capstone Receipt: The Full Pipeline

The Complete Commit-then-Simulate (End-to-End)

```
10 ENCRYPTED envelopes arrive (nobody can read them)
  → ORDER COMMITTED (root=7ec2cd4c..., beacon-shuffled)
    → 10 THRESHOLD DECRYPTS (all tag OK)
      [decrypt] envelope 0: "encrypted-decree-0" (tag OK)
      [decrypt] envelope 1: "encrypted-decree-1" (tag OK)
      ... through envelope 9
    → 10 FOLDS (ALL SAT)
      [fold] decree 1/10 folded, SAT
      [fold] decree 2/10 folded, SAT
      ... through decree 10/10
    → _E epoch 1: wrap_verify ACCEPT (stage=0)
    → PoUW epoch 1: 643 GEMM ACCEPT, weight 262144 MACs
```

Corollary 25 (The Four-Pillar Integration). *The complete Commit-then-Simulate pipeline — encrypted in, ordered, decrypted, folded, π_E produced, PoUW weight derived — is **structurally guaranteed** by the four pillars operating as a single system. Front-running is impossible because the ordering is committed before any party can read the payloads, and the threshold controls who can decrypt.*

6.8 The Endianness Lesson

During the envelope implementation, a subtle defect was caught: the serializer writes G2 coordinates in **little-endian per limb**, but the deserializer read **big-endian**. The deserialized point was valid-looking (non-zero coordinates) but mathematically a different point. The R-roundtrip check (serialize → deserialize → re-serialize, compare) caught it immediately.

The Serialization Law

A serialization format's byte order is part of its contract. The serializer and deserializer must agree per limb, not just per structure. This principle applies to every wire format in the system.

7 Pillar 4: Holographic State Folding

7.1 The Folding Stack

- **Primary:** HyperNova over Pallas ($\mathbb{F}_p$), CCS constraint system
- **Auxiliary:** CycleFold over Vesta ($\mathbb{F}_q$) for cross-curve commitment digests
- **Non-uniform IVC:** SuperNova-style NIVC over $\mathcal{F} = \{F_{\text{head}}, F_{\text{exec}}, F_{\text{close}}\}$
- **Program counter:** constrained variable with explicit transition matrix (head → exec⁺ → close; all other edges unsatisfiable)
- **Accumulator:** 41 bytes, $O(1)$ in transaction count

7.2 The Fold Circuit Constraints

Per decree entry ℓ with certified tuple $(ct_hash_\ell, pt_hash_\ell, status_\ell)$:

- **Binding:** $a \cdot (\text{Poseidon}(\text{HSM_PT_v1}, pt_\ell) - pt_hash_\ell) = 0$
- **Nonce:** $a \cdot (\text{leaf_nonce}(\text{sender}_\ell) + 1 - pt.\text{nonce}) = 0$
- **Balance:** Sender debit, recipient credit via canonical registers with output range checks
- **Fee underflow:** Explicit LT gate: $\text{balance} \geq \Delta_{\text{req}}$ before subtraction
- **Self-send:** If sender = recipient, assert `recipient_pre == sender_post`
- **PAD neutrality:** PAD rows bypass state verification entirely

7.3 The Certificate Assumption

The circuit assumes `decree_root` is valid; the verifier checks the threshold-BLS certificate out-of-circuit. In-circuit pairing verification would cost $\geq 10^6$ constraints.

7.4 The π_E Receipt

	hsmaPurple!20 Component	Size (bytes)
2hsmaPurple!5white	T1 (opening transcript)	544
	T2 (batched fold transcript)	768
	OOD (8 points + commitment)	288
	hsmaGreen!15 Total π_E	1,600
	Budget	73,728
	Utilization	2.2%

7.5 Production-Scale Projection

	hsmaPurple!20 Component	Constraints
2hsmaPurple!5white	f_{exec} transition (8×476)	3,808
	Poseidon hash ($1,088 \times 3 \times 476$)	1,553,664
	SMT opening ($5,120 \times 2 \times 476$)	4,874,240
	Total	6,431,712
	10^6 target	Exceeded (6.4×)

7.6 Proof-Carrying Data Boundary

Epoch proof π_E chains via `prev_digest` absorption in F_{head} . Light clients download the epoch header (`height, state_root,  $\pi_E$ , BLS_cert`), verify the BLS signature (1 pairing), and verify π_E (HyperNova verifier, $O(1)$, ≈ 40 ms phone-class).

7.7 The Async Fold Pipeline

Fold pipeline is asynchronous: $T_{\text{fold}} = 20\%$ of epoch. First certified π_E publisher earns a reward. Breach triggers alarms; the chain continues on full nodes — proof availability $\neq$ chain liveness.

Part IV Security Analysis

8 End-to-End Security: The ϵ -Budget

8.1 The A6 Composition Theorem

The Total System Soundness

$$\epsilon_{\text{sys}} = \underbrace{0}_{\text{L1: Quorum forgery}} + \underbrace{0}_{\text{L2: Committee takeover}} + \underbrace{0}_{\text{L4: Eclipse divergence}} + \underbrace{2^{-128}}_{\text{L5: PCS soundness}} + \underbrace{2^{-127}}_{\text{L6: NIFS completeness}} + \underbrace{2^{-171}}_{\text{Hash collision}} \quad (16)$$

8.2 The Hypotheses

hsmaBlue!20 Hypothesis	2hsmaBlue!5white Statement
H1	BLS12-377 threshold-BLS EUF-CMA $\leq 2^{-110}$; co-CDH; Poseidon idealization; DDH+AEAD
H2	Honest preference consistency on common views
H3	Conflict propagation within $T_{\text{close}} < T_{\text{stall}}$
H4	Snapshot authenticity, staleness ≤ 1 epoch = 600 s
H5	Shipped circuits $\equiv$ these specifications

8.3 The Structural Zeros: Engineered, Not Inherent

- **L1 (Quorum forgery):** $\epsilon = 0$ by authentication + frozen snapshots + $\varphi_{\text{floor}} > f/\alpha$. Remove the floor and the bound degrades.
- **L2 (Committee takeover):** $\epsilon = 0$ by the identity cap: adversary seats $\leq 100 < t = 112$. Remove the dust floor and the cap dissolves.
- **L4 (Eclipse divergence):** $\epsilon = 0$ by INV-G3 containment. Remove the path diversity and eclipse becomes a divergence vector.

The Parameter Mandate

Remove the structural floor and the sortition term degrades to $\approx 2^{-73}$ (dust-spreading Poisson limit) — below the 2^{-80} target. The zeros are a **parameter mandate**, not a gift. Each parameter is enforced in code, checked by the CI, and pinned in the genesis.

8.4 The Annualized Bound

$$\epsilon_{\text{year}} \leq 52,596 \cdot 2^{-110} \approx 2^{-94.3} \quad (17)$$

The 2^{-80} design target is cleared annually with 14 bits of margin.

Part V The Engineering Discipline

9 The Bilingual Golden-Oracle Methodology

9.1 The Four Components

1. **The Python Oracle:** an independent implementation of every cryptographic primitive in Python, using arbitrary-precision integers. No Montgomery arithmetic, no fixed-width limitations, no implementation shortcuts. The oracle serves as the mathematical reference.
2. **The C++ Implementation:** the production implementation using fixed-width Montgomery arithmetic for performance.
3. **Golden Vectors:** emitted by the Python oracle, containing every intermediate value for a set of test inputs. These vectors pin the expected output at each step.
4. **The Gate:** an automated verification pipeline that regenerates all goldens, compiles all C++ modules from source, runs all conformance tests against the fresh goldens, and aborts on any failure.

Theorem 26 (Divergence Detection). *Let I_1 and I_2 be two implementations sharing no code path. For any x in the test set T , if $I_1(x) \neq I_2(x)$, the divergence is detected with probability 1.*

Proof. By construction: the golden vector pins I_2 's output for each $x \in T$. I_1 computes independently. The gate compares the two outputs unconditionally. If they differ, the gate reports FAIL and aborts. $\square$

9.2 The Verification Gate Pipeline

1. **Regenerate:** execute the golden-vector emitter (Python) to produce all 41 header files
2. **Configure:** CMake configure with Ninja
3. **Build:** compile all test binaries (abort on failure)
4. **Lint:** check for floating-point tokens (the float-free law)
5. **FAILED-target tripwire:** any FAILED build target fails the gate
6. **Freshness:** verify binary timestamps vs. golden timestamps
7. **Diagnostics:** run structural health checks
8. **Conformance:** run all 34 test binaries
9. **Verdict:** GATE GREEN if and only if all steps pass

	hsmaGold!20 Metric	Value
	Decisions in the append-only ledger	281
	Defects owned with named fixes	223
	Generalizable laws extracted	203
	Conformance gates	34/34
2hsmaGold!5white	Generated headers (determinism-proven)	41
	Diagnostic probes	30+
	Testnet nodes	2
	Encrypted envelopes processed	10
	Verified MACs per epoch	262,144
	Epoch proof size	1,600 bytes

9.3 The Engineering Culture in Numbers

9.4 Selected Engineering Narratives

The htonl phantom (six sessions silent): A missing `htonl()` caused the node's connect function to dial the byte-swapped address. The log printed the correct IP while the socket dialed wrong. Six test runs passed because the connections were implicitly not established, not explicitly failing. Found by eliminating every other hypothesis.

The Law of Printed Correctness

Printed correctness is not wire correctness. A field's display path and its use path are verified separately. A log line that shows the right value says nothing about the bytes that actually crossed the wire.

The stale assertion (six gates asleep): A conformance test *encoded* a known collision as correct behavior. The forced rebuild (from the migration) exposed the contradiction.

The Supersedes Law

A supersedes claim triggers a consumer audit of the superseded convention. Every assertion that references a changed behavior must be re-verified against the new behavior.

The endianness seam: The serializer writes G2 coordinates in little-endian per limb; the decoder read big-endian. The deserialized point was valid-looking but mathematically wrong. Found by the R-roundtrip check.

The Serialization Law

A serialization format's byte order is part of its contract. The serializer and deserializer must agree per limb, not just per structure. A valid-looking deserialization that produces a different point is a correctness failure, not a formatting preference.

Part VI Reproducibility and the Path Forward

10 Reproducibility

```
git clone https://github.com/sarinsk629-blip/hsma-core.git
cd hsma-core
./scripts/gate.sh
# 41 goldens generated (determinism-proven: two-run byte-identical)
# 34/34 tests passed
# GATE GREEN
```

The verification is reproducible by anyone, on any hardware, with zero external dependencies. The 41 golden headers are tracked in the repository (determinism-proven: two-run byte-identical), and every constant change is a reviewable diff.

11 The Path Forward

1. **Testnet growth:** from 2 nodes to 5+ nodes, with the 24/7 AWS validator
2. **Aggregate-verify integration:** one pairing per vote batch instead of N (the $3.0\times$ speedup, measured)
3. **DKG rotation:** live committee regeneration per epoch
4. **Full Poseidon in the epoch loop:** the 1,088-constraint gadget replaces the reduced version
5. **External audit:** an independent cryptographic security review
6. **Mainnet preparation:** token economics, final audit, launch

12 Conclusion

The Complete System

HSMA is a **folding-based state machine** where:

- Consensus weight derives from **verified AI inference** — not wasted energy, not capital
- The mempool is **cryptographically unreadable** before ordering — zero-MEV is structural, not economic
- The entire epoch history compresses into a **1,600-byte recursive proof** — verifiable without the witness
- Every design decision, every defect, every fix is **owned in an append-only ledger** — zero hiding

The four pillars are structurally interdependent: encrypted envelopes enter the mempool, the ordering ceremony commits the sequence, the threshold decrypt unlocks the payloads, the fold compresses them into π_E , the PoUW derives consensus weight from the verified computation, and the MSSC resolves any disagreement through the weight-weighted sampling loop.

The system is implemented, tested, and receipted.

The Satoshi path: build → publish → verify → wait.

Built by one person. Zero hiding.

**281 decisions. 223 defects owned. 203 laws.
34 gates green. 41 goldens. One unified system.**

Not a blockchain. Not a DAG. A new category.

A The Complete Domain Separation Registry

Role	Tag / IV
State tree nodes	IV_STATE_NODE
State tree leaves	IV_STATE_LEAF
Order tree	IV_ORDER
Decree tree	IV_DECREE
Weight tree	IV_WEIGHT
Contact root	IV_CONTACT
Revocation root	IV_REVOCATION
Vote preimage	HSM_MSSC_VOTE_v1
Ordering lock	HSM_ORDER_v1
Decryption share	HSM_DEC_SHARE_v2
Epoch beacon	HSM_BEACON_V1
Hash-to-G1	HSM_H2G1_V1
M2 key derivation	HSM_KDF_V1
M2 DEM keystream	HSM_DEM_V1
M2 DEM tag	HSM_DEM_TAG_V1
M2 ciphertext hash	HSM_CT_V1
M2 order root	HSM_ORDROOT_V1
M2 user auth	HSM_SIG_USER_V1
Sum-check transcript	HSM_SUMCHECK_v1
CycleFold hash	HSM_CYCLEFOLD_v1
PoUW transcript	HSM_POUW_v1
Peer seed	HSM_PEERSEED_v1
PT binding	HSM_PT_v1
Account derivation	HSM_ACCT
Vesta state node	IV_STATE_NODE_V
Vesta state leaf	IV_STATE_LEAF_V
Vesta decree tree	IV_DECREE_V

B Repository Structure

```
hsma-core/  
    include/hsma/          # 41 implementation headers  
        fe.hpp             # Pallas base field  
        poseidon.hpp       # Poseidon-3 hash  
        fev.hpp            # Vesta base field  
        poseidon_v.hpp     # Vesta-domain Poseidon  
        vault.hpp          # SMT state vault  
        consensus.hpp      # MSSC automaton  
        msscloop.hpp       # Live sampling loop
```

```

msscvote.hpp          # Vote wire layer
m2envelope.hpp        # Encrypted envelope
pouw.hpp              # PoW verifier
gkr.hpp               # GEMM circuit
mssc + fold + sumcheck + pcs + nifs + whir + ...
threshold/            # BLS12-377 stack (15 headers)
conformance/          # 34 test binaries
tools/                # 30+ probes + faucet + votecast
scripts/              # gate.sh + generators
generated/            # 41 golden headers (tracked)
docs/                 # ledger + whitepaper + eprint
attic/                # historical artifacts
third_party/          # reference crates
.github/workflows/    # CI gate

```

C Test Coverage Map

Test	What It Proves
step2	Field arithmetic (2,472 goldens)
step3	Numeric kernel (scratch127)
step4	SMT state vault
step5	Mempool semantics
step6	MSSC automaton
step7	BLS12-377 F _r kernel
step8	G1 promotion parity
step9	Threshold protocol (h2g1, DKG, beacon)
step10	F _{q2} + twist + G2
step10b	Pairing: Miller, Tate, BLS law
step12	M2 order-bound decryption
step13	M2 production hardening
step14	Fold step family
step15	CCS constraint layer
step16	NIVC fold accumulator
step17	Epoch pipeline
step18	G1 transport (sampler)
step19	Sum-check engine
step20	Multilinear PCS
step21	NIFS fold
step22	Light-client certificate
step23	End-to-end epoch (ϕ_0 - ϕ_7)
step24	Vesta field twin
step25	Vesta-domain Poseidon
step26-34	Pasta cycle + folding + PoW